

Predictive Modeling Using Machine Learning

Project Title: Employee Salary Prediction and Classification Using Machine Learning

A complete supervised machine learning project covering regression, classification, model evaluation, visualization, and prediction.

1. Project Overview

This project uses supervised machine learning to predict an employee's salary based on factors such as experience, education, job level, working hours, performance, previous salary, and number of projects. It implements Linear Regression, Decision Tree Regressor, Random Forest Regressor, and Random Forest Classification. The project also evaluates the models and produces visualizations such as model comparison, actual-versus-predicted plots, feature importance, and a confusion matrix.

2. Problem Statement

Employee salary depends on several factors. Manually estimating salary is difficult when many variables influence compensation. The proposed system learns patterns from historical employee data and predicts an expected salary for a new employee. A second classification task categorizes employees into Low, Medium, or High salary groups.

3. Objectives

- Collect and prepare employee data.
- Clean missing and duplicate values.
- Perform exploratory data analysis.
- Select important features.
- Train multiple supervised learning algorithms.
- Test and compare trained models.
- Predict salary for new employees.
- Classify employees into salary categories.
- Evaluate classification using a confusion matrix and classification report.
- Visualize model performance and feature importance.

4. Technologies Used

Technology	Purpose
Python	Programming
Pandas	Data processing
NumPy	Numerical operations
Matplotlib	Visualization
Seaborn	Statistical visualization
Scikit-learn	Machine learning
Joblib	Saving trained models

5. Dataset

Create a file named **employee_salary.csv**. Suggested columns:

Column	Description
Age	Employee age
Experience	Years of experience
Education	Education level
Job_Level	Employee job level
Performance	Performance score
Working_Hours	Average weekly working hours
Previous_Salary	Previous salary
Projects	Number of completed projects
Salary	Current annual salary

Sample records:

Age, Experience, Education, Job_Level, Performance, Working_Hours, Previous_Salary, Projects, Salary
23,1,1,1,3,40,25000,2,30000
25,2,2,1,4,42,30000,3,38000
28,4,2,2,4,44,40000,5,52000
30,6,3,2,5,45,52000,7,68000
35,10,3,3,5,46,70000,10,95000

6. Machine Learning Workflow

```
Employee Dataset
↓
Data Cleaning
↓
Exploratory Data Analysis
↓
Feature Selection
↓
Data Preprocessing
↓
Train/Test Split
↓
Linear Regression / Decision Tree / Random Forest
↓
Model Evaluation
↓
Model Comparison
↓
Salary Prediction
↓
Salary Classification + Confusion Matrix
```

7. Complete Regression Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("employee_salary.csv")
print(df.head())
print(df.info())
print(df.isnull().sum())

df = df.drop_duplicates()

numeric_columns = df.select_dtypes(include=np.number).columns
for column in numeric_columns:
    df[column] = df[column].fillna(df[column].median())

plt.figure(figsize=(10, 6))
sns.histplot(df["Salary"], bins=20, kde=True)
```

```

plt.title("Salary Distribution")
plt.xlabel("Salary")
plt.ylabel("Number of Employees")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x="Experience", y="Salary")
plt.title("Experience vs Salary")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 7))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.tight_layout()
plt.show()

X = df[[
    "Age", "Experience", "Education", "Job_Level",
    "Performance", "Working_Hours", "Previous_Salary", "Projects"
]]
y = df["Salary"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
linear_prediction = linear_model.predict(X_test)

tree_model = DecisionTreeRegressor(max_depth=6, random_state=42)
tree_model.fit(X_train, y_train)
tree_prediction = tree_model.predict(X_test)

forest_model = RandomForestRegressor(
    n_estimators=100, random_state=42
)
forest_model.fit(X_train, y_train)
forest_prediction = forest_model.predict(X_test)

def evaluate_model(name, actual, predicted):
    mae = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))
    r2 = r2_score(actual, predicted)

    print("\n=====")
    print(name)
    print("=====")
    print("MAE :", round(mae, 2))
    print("RMSE:", round(rmse, 2))
    print("R2 Score:", round(r2, 4))
    return r2

linear_r2 = evaluate_model("Linear Regression", y_test, linear_prediction)
tree_r2 = evaluate_model("Decision Tree", y_test, tree_prediction)
forest_r2 = evaluate_model("Random Forest", y_test, forest_prediction)

comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Decision Tree", "Random Forest"],
    "R2_Score": [linear_r2, tree_r2, forest_r2]
})
print(comparison)

plt.figure(figsize=(9, 6))
sns.barplot(data=comparison, x="Model", y="R2_Score")
plt.title("Model Performance Comparison")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 6))
plt.scatter(y_test, forest_prediction)
plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title("Actual Salary vs Predicted Salary - Random Forest")
plt.tight_layout()
plt.show()

importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": forest_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

```

```
print(importance_df)

plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df, x="Importance", y="Feature")
plt.title("Random Forest Feature Importance")
plt.tight_layout()
plt.show()

new_employee = pd.DataFrame({
    "Age": [27],
    "Experience": [4],
    "Education": [2],
    "Job_Level": [2],
    "Performance": [4],
    "Working_Hours": [43],
    "Previous_Salary": [45000],
    "Projects": [5]
})

prediction = forest_model.predict(new_employee)
print("Predicted Salary:", round(prediction[0], 2))
```

8. Model Evaluation

MAE: Mean Absolute Error represents the average absolute difference between actual and predicted salary. **RMSE:** Root Mean Squared Error gives greater weight to larger errors. **R² Score:** Measures how much of the variation in salary is explained by the model.

Because salary prediction is a regression problem, confusion matrices and ROC curves are not directly appropriate for the regression target. To satisfy the classification-evaluation requirement, the project adds a second task that classifies employees into Low, Medium, and High salary categories.

9. Salary Classification

```
df["Salary_Category"] = pd.cut(
    df["Salary"],
    bins=[0, 40000, 70000, float("inf")],
    labels=["Low", "Medium", "High"]
)
```

10. Confusion Matrix Classification Code

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split

X = df[[
    "Age", "Experience", "Education", "Job_Level",
    "Performance", "Working_Hours", "Previous_Salary", "Projects"
]]
y = df["Salary_Category"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

classifier = RandomForestClassifier(
    n_estimators=100, random_state=42
)

classifier.fit(X_train, y_train)
prediction = classifier.predict(X_test)

cm = confusion_matrix(y_test, prediction)

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Salary Classification Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print(classification_report(y_test, prediction))
```

11. Project Folder Structure

```
Predictive-Modeling-ML/
├── data/
│   └── employee_salary.csv
├── src/
│   ├── regression.py
│   └── classification.py
├── outputs/
│   ├── salary_distribution.png
│   ├── experience_salary.png
│   ├── correlation_matrix.png
│   ├── model_comparison.png
│   ├── feature_importance.png
│   └── confusion_matrix.png
├── notebooks/
│   └── predictive_modeling.ipynb
├── requirements.txt
├── README.md
└── LICENSE
```

12. requirements.txt

```
pandas  
numpy  
matplotlib  
seaborn  
scikit-learn  
joblib
```

13. Installation

```
pip install pandas numpy matplotlib seaborn scikit-learn joblib
```

14. Expected Project Results

- ✓ Cleaned employee dataset
- ✓ Exploratory data analysis and visualizations
- ✓ Linear Regression model
- ✓ Decision Tree model
- ✓ Random Forest model
- ✓ MAE, RMSE, and R^2 evaluation
- ✓ Model performance comparison
- ✓ Actual versus predicted salary plot
- ✓ Random Forest feature importance
- ✓ Salary classification
- ✓ Confusion matrix and classification report
- ✓ New employee salary prediction

15. Future Enhancements

- Build a Streamlit web dashboard.
- Save trained models with Joblib.
- Add a larger real-world employee dataset.
- Use hyperparameter tuning with GridSearchCV.
- Add cross-validation.
- Deploy the model as a web application.
- Add an interactive employee salary prediction form.

Final Project Title: Predictive Modeling of Employee Salary Using Supervised Machine Learning